

PRACTICAL 4

PRACTICAL TITLE :- JavaScript DOM Manipulation, Event Handling, and UI Interactivity

Problem Definition Add dynamic UI components such as collapsible FAQ, modal popup, image/content slider, notification banner, hamburger menu, and light/dark theme switcher using JavaScript.

Key Questions / Analysis
/ Interpretation

- 1. How are DOM elements selected and modified?**
- 2. Are event listeners attached correctly?**
- 3. Is localStorage used for remembering theme choice?**
- 4. Does interactivity improve usability without breaking accessibility?**

Key Skills to be
Addressed

DOM selection, event handling, class manipulation, localStorage, basic animation

Applications Interactive dashboards and dynamic frontend behaviour

Learning Outcome Students will apply JavaScript to enhance user experience and client-side

interactivity.

Dataset/Test Data Static FAQ/event content

Tools/Technology to be

Used

JavaScript ES6+, HTML5, CSS3, browser dev tools

Total Hours Implementation: 4 hours | Total engagement: 6 hours

Post Laboratory Work

Description

Submit live demo and source-code explanation.

Evaluation Strategy

Including Viva

DOM logic, event handling correctness, usability, and browser console testing.

Advanced/Intermediate

Extension

Intermediate: Add transition effects to cards/buttons.

Advanced: Store user UI preferences in localStorage and restore them on page load.

GIT HUB REPOSITORY LINK :-

<https://github.com/nehhalvala0005-cloud/sh.git>

1. Aim

To implement JavaScript DOM manipulation, event handling, and UI interactivity by creating dynamic components such as a collapsible FAQ, modal popup, image/content slider, notification banner, hamburger menu, and light/dark theme switcher.

2. Problem Definition

Add dynamic UI components such as:

- Collapsible FAQ
- Modal popup
- Image/content slider
- Notification banner
- Hamburger menu
- Light/Dark theme switcher

Use JavaScript to manipulate DOM elements and handle user interactions.

3. Key Questions / Analysis

1. How are DOM elements selected and modified?
 2. Are event listeners attached correctly?
 3. Is localStorage used for remembering the theme choice?
 4. Does interactivity improve usability without breaking accessibility?
-

4. Key Skills / Concepts Used

- **DOM Selection**
 - **DOM Manipulation**
 - **Event Handling**
 - **addEventListener()**
 - **Class Manipulation**
 - **classList**
 - **localStorage**
 - **JavaScript ES6+**
 - **Basic CSS Animation**
-

5. Tools / Technologies Used

- **HTML5**
 - **CSS3**
 - **JavaScript ES6+**
 - **Visual Studio Code**
 - **Web Browser**
 - **Browser Developer Tools**
-

6. Theory

DOM Manipulation

DOM stands for Document Object Model. It represents an HTML document as a tree of objects. JavaScript can select and modify these objects dynamically.

Example:

```
document.getElementById("title").textContent = "StudentHub";
```

Event Handling

JavaScript events allow a webpage to respond to user actions such as:

- **Click**

- **Mouse movement**
- **Keyboard input**
- **Form submission**

Example:

```
button.addEventListener("click", function() {  
    alert("Button clicked!");  
});
```

localStorage

localStorage allows data to be stored in the browser so that it can be used even after refreshing the page.

Example:

```
localStorage.setItem("theme", "dark");
```

7. Implementation

The following components were implemented using HTML, CSS and JavaScript:

A. Collapsible FAQ

When the user clicks a question, the answer is displayed or hidden.

B. Modal Popup

A popup window is displayed when the user clicks a button.

C. Content Slider

Users can move between different pieces of content using navigation buttons.

D. Notification Banner

A notification message is displayed to inform the user about an update.

E. Hamburger Menu

A navigation menu is opened and closed using a menu button, especially useful on smaller screens.

F. Light/Dark Theme Switcher

The user can switch between light and dark themes. The selected theme is stored using localStorage.

8. Sample JavaScript Code

```
const faqButton = document.getElementById("faqButton");  
const faqAnswer = document.getElementById("faqAnswer");
```

```
faqButton.addEventListener("click", function() {  
    faqAnswer.classList.toggle("show");  
});
```

```
const modal = document.getElementById("modal");  
const openModal = document.getElementById("openModal");  
const closeModal = document.getElementById("closeModal");
```

```
openModal.addEventListener("click", function() {  
    modal.style.display = "flex";  
});
```

```
closeModal.addEventListener("click", function() {  
    modal.style.display = "none";  
});
```

```
const menuButton = document.getElementById("menuButton");
```

```
const menu = document.getElementById("menu");
```

```
menuButton.addEventListener("click", function() {  
    menu.classList.toggle("active");  
});
```

```
const themeButton = document.getElementById("themeButton");
```

```
themeButton.addEventListener("click", function() {
```

```
    document.body.classList.toggle("dark");
```

```
    if (document.body.classList.contains("dark")) {
```

```
        localStorage.setItem("theme", "dark");
```

```
    } else {
```

```
        localStorage.setItem("theme", "light");
```

```
    }
```

```
});
```

```
window.addEventListener("load", function() {
```

```
    const savedTheme = localStorage.getItem("theme");
```

```
    if (savedTheme === "dark") {  
        document.body.classList.add("dark");  
    }  
  
});
```

9. Algorithm

Step 1

Create the required HTML elements for FAQ, modal, slider, notification, menu and theme switcher.

Step 2

Select the required DOM elements using JavaScript.

Step 3

Attach event listeners to buttons and interactive elements.

Step 4

Use `classList` to add or remove CSS classes dynamically.

Step 5

Use JavaScript to display and hide the modal and FAQ content.

Step 6

Implement the hamburger menu using class manipulation.

Step 7

Implement light/dark theme switching.

Step 8

Store the selected theme in `localStorage`.

Step 9

Restore the saved theme when the webpage is loaded.

Step 10

Test all interactive components using the browser.

10. Expected Output

The webpage should provide the following functionality:

Component	Expected Behaviour
FAQ	Answer opens/closes on click
Modal	Popup opens and closes
Slider	Content changes using navigation
Notification	Notification banner is displayed
Hamburger Menu	Navigation opens/closes
Theme Switcher	Light/Dark mode changes
localStorage	Theme remains after page refresh

11. Testing

Test Case 1 – FAQ

Action: Click FAQ question

Expected: Answer should appear.

Test Case 2 – Modal

Action: Click Open button

Expected: Modal popup should appear.

Test Case 3 – Hamburger Menu

Action: Click menu button

Expected: Navigation menu should open/close.

Test Case 4 – Theme Switcher

Action: Click theme button

Expected: Page theme should change.

Test Case 5 – Theme Persistence

Action: Select dark theme and refresh page

Expected: Dark theme should remain selected.

12. Result

The JavaScript DOM manipulation and event handling features were successfully implemented. Interactive UI components such as FAQ, modal popup, content slider, notification banner, hamburger menu and light/dark theme switching were demonstrated.

The use of localStorage successfully allows the selected theme preference to be remembered after page refresh.

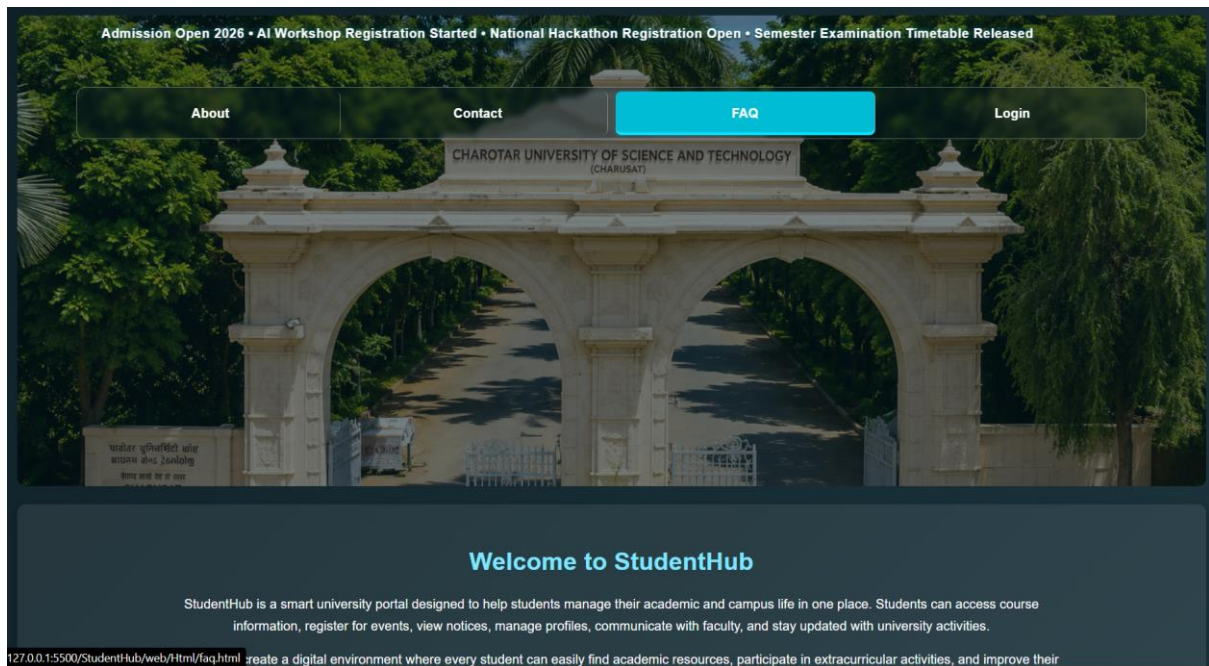
13. Learning Outcome

After completing this practical, I learned how to:

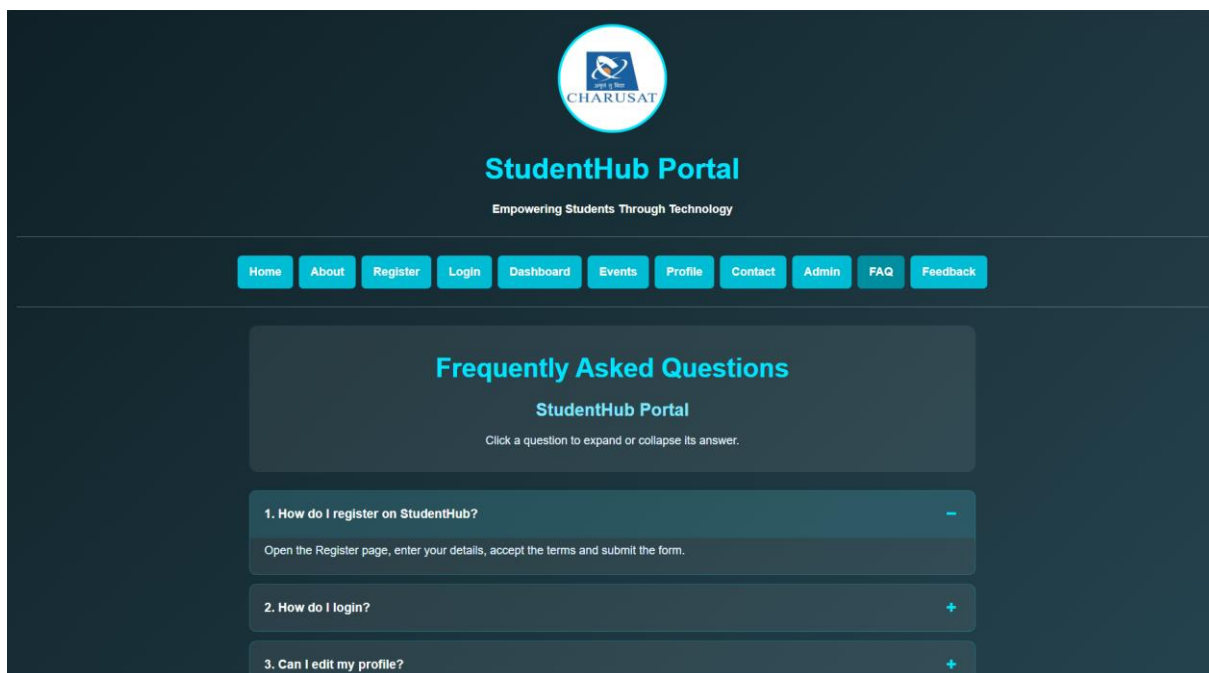
- Select and modify DOM elements using JavaScript.
- Handle user events using event listeners.
- Dynamically add and remove CSS classes.
- Create interactive UI components.
- Use localStorage to store user preferences.
- Improve frontend usability using JavaScript.

PROOF OF WORK :-

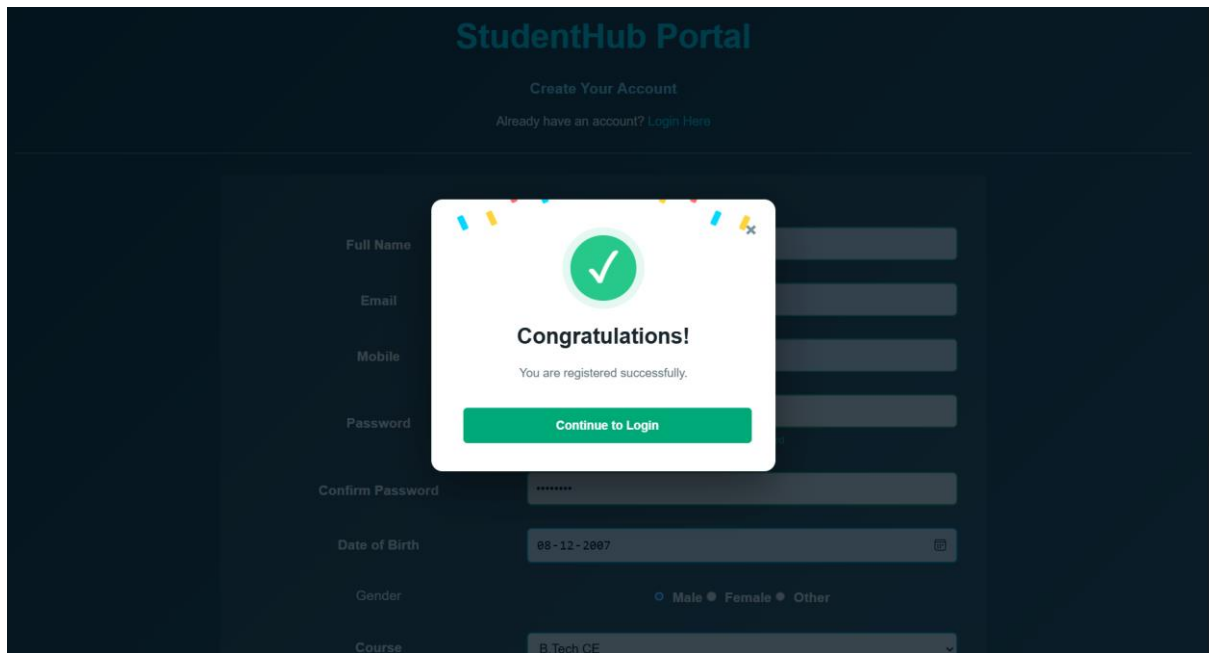
4.1 :- Normal StudentHub Page



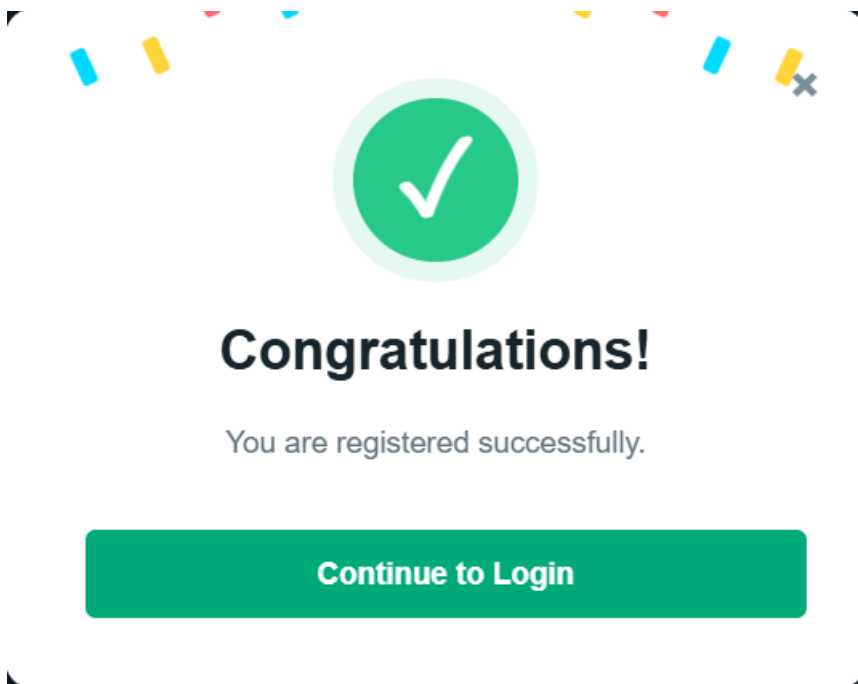
4.2 :- FAQ open



4.3 :- Modal Popup



4.4 :- Notification Banner



Learning Outcomes

After completing this practical, I was able to:

- Understand the basics of JavaScript DOM manipulation.
- Implement event handling for interactive webpage elements.
- Create and manage dynamic popups and notifications.
- Improve webpage user interaction and responsiveness using JavaScript.
- Understand how JavaScript can dynamically modify HTML elements and their properties.
- Develop a better understanding of creating interactive web applications.

Conclusion

Thus, JavaScript DOM manipulation and event handling were successfully implemented. Interactive components were created to make the StudentHub webpage more user-friendly and responsive. The practical helped in understanding how JavaScript can dynamically control webpage elements and respond to user actions.

Faculty Remark :- _____